

Guía completa del proyecto

Nodo LoRa (Heltec LoRa32u4 v1.1) con sensores LDR + DHT11 + IR
Integración con TTN + MQTT + Node-RED (Windows 11) y Dashboard

Autor: Joel Cañete

Fecha: 15/01/2026

Versión: 1.0

Tabla de contenido

1. Resumen del proyecto
2. Materiales y componentes
3. Conexiones eléctricas
4. Configuración Arduino IDE
5. Código del nodo LoRa
6. Configuración en The Things Network (TTN)
7. Integración MQTT
8. Instalación y configuración de Node-RED (Windows 11)
9. Dashboard en Node-RED

1. Resumen del proyecto

Este documento describe paso a paso la implementación de un nodo IoT basado en LoRaWAN utilizando la placa Heltec LoRa32u4 v1.1. El nodo mide nivel de luz (LDR), temperatura y humedad (DHT11) y detección por infrarrojo (emisor-receptor con salida digital). Los datos se envían por LoRaWAN mediante OTAA a The Things Network (TTN) y luego se consumen por MQTT e integración en Node-RED para visualización en dashboard.

1.1 Alcance

Incluye: hardware, cableado, configuración de Arduino IDE, código, TTN (aplicación + dispositivo + decoder), MQTT, Node-RED (instalación en Windows 11) y dashboard.

1.2 Arquitectura general

Flujo de datos: Sensores → Heltec LoRa32u4 → LoRaWAN OTAA → TTN → MQTT → Node-RED → Dashboard

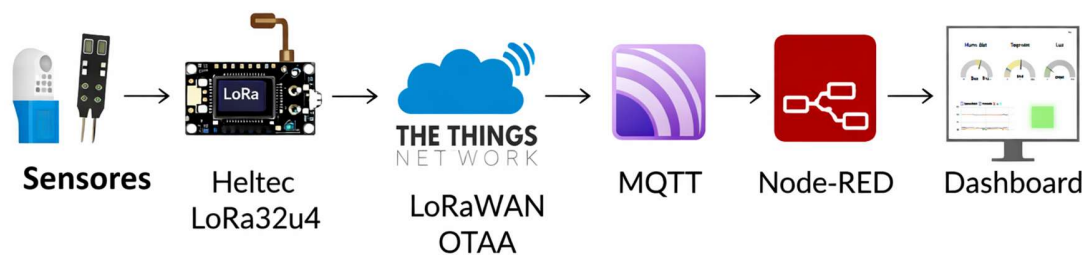


Figura 1: Diagrama de arquitectura

2. Materiales y componentes

Componentes principales:

- Placa Heltec LoRa32u4 v1.1 (ATmega32u4 + SX1276).
- Sensor LDR (módulo o divisor resistivo) conectado a entrada analógica A0.
- Sensor DHT11 (3 pines: VCC, GND, DATA).
- Sensor IR emisor–receptor (3 pines: VCC, GND, OUT) – tipo detección de obstáculo/barrera.
- Cables dupont, protoboard, resistencia (si el LDR no viene en módulo) y fuente/alimentación.
- PC con Windows 11 + Arduino IDE + Node.js + Node-RED.
- Gateway LoRaWAN compatible con TTN (US915 en Paraguay).

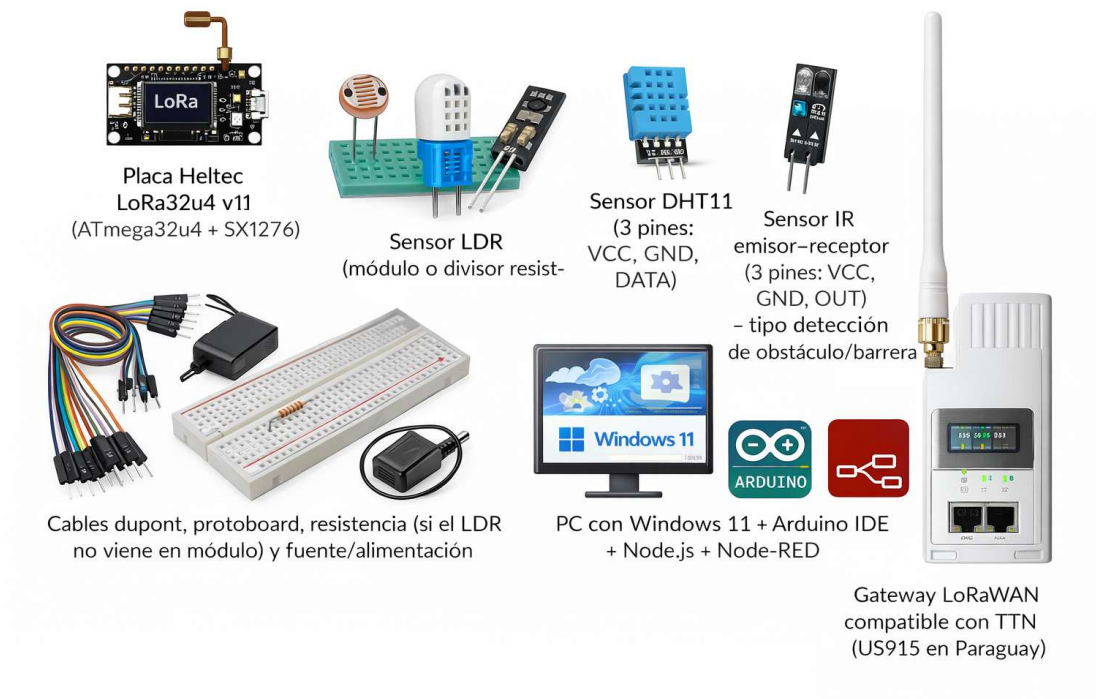


Figura 2: Foto de los componentes.

3. Conexiones eléctricas

Las conexiones propuestas a continuación están pensadas para no interferir con el módulo LoRa

Dispositivo	Pin del sensor	Pin en Heltec LoRa32u4 v1.1
LDR	OUT/Señal	A0
LDR	VCC	3.3V o 5V (según módulo)
LDR	GND	GND
DHT11	DATA	D2
DHT11	VCC	5V (o 3.3V si el módulo lo admite)
DHT11	GND	GND
IR (3 pines)	OUT	D3
IR (3 pines)	VCC	5V
IR (3 pines)	GND	GND

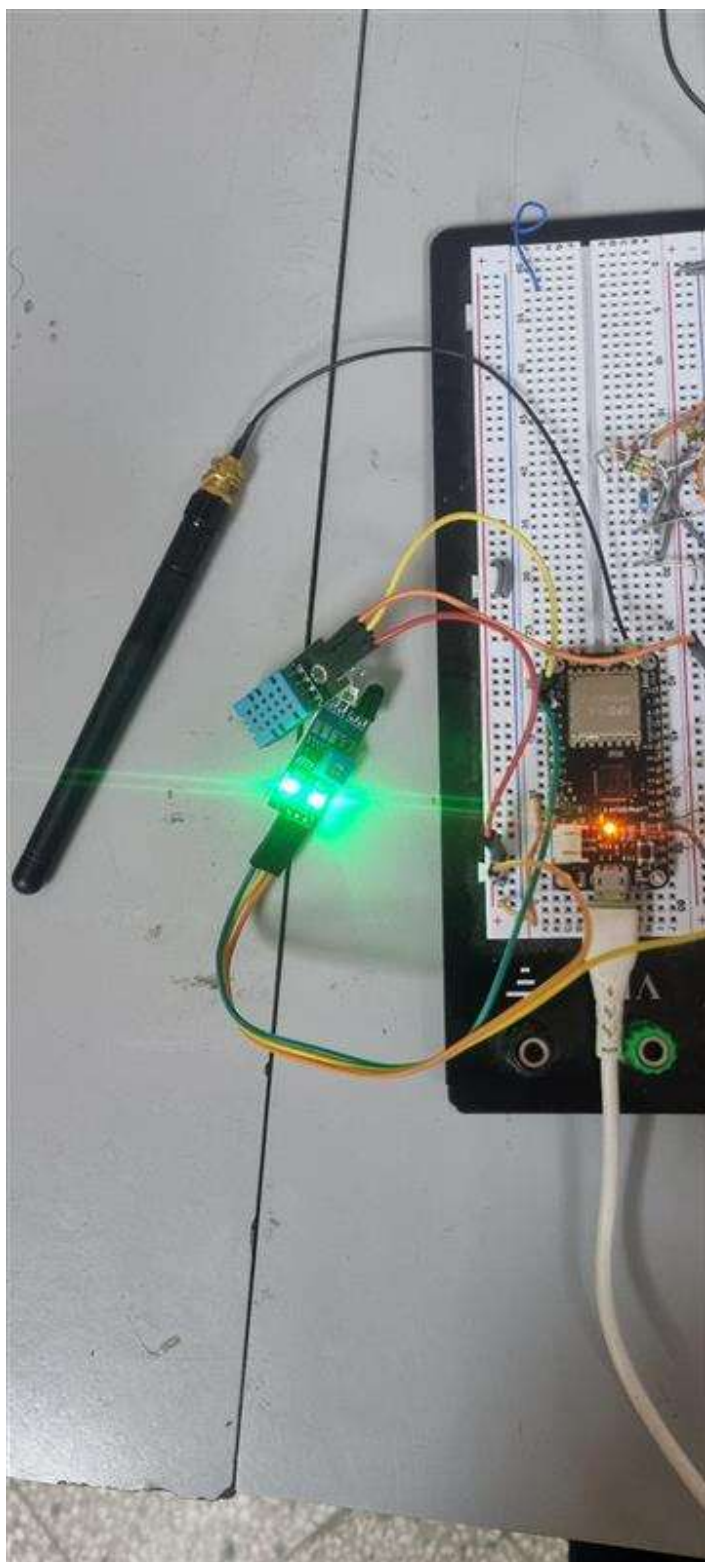


Figura 3: Foto del cableado en protoboard

3.1 Notas importantes

- Verificar el nivel lógico del pin OUT del IR: muchos módulos entregan LOW cuando detectan objeto.
- El DHT11 requiere un intervalo de lectura ≥ 2 s; se recomienda enviar por LoRaWAN cada 60 s o más para evitar saturación y reducir consumo.
- Evita usar pines reservados para LoRa (según el pinmap configurado en LMIC).

4. Configuración de Arduino IDE

Esta sección documenta la instalación del entorno y librerías necesarias para compilar y cargar el firmware al Heltec LoRa32u4.

4.1 Instalación de Arduino IDE

- 1) Descargar e Instalar Arduino IDE. Una vez instalado vemos en que puerto COM nos habilito la placa con la PC, en mi caso habilito el com 12 por lo que seleccionamos ese puerto y luego seleccionamos la placa que vamos a utilizar
- 2) Seleccionar la placa/puerto correspondiente al Heltec LoRa32u4.

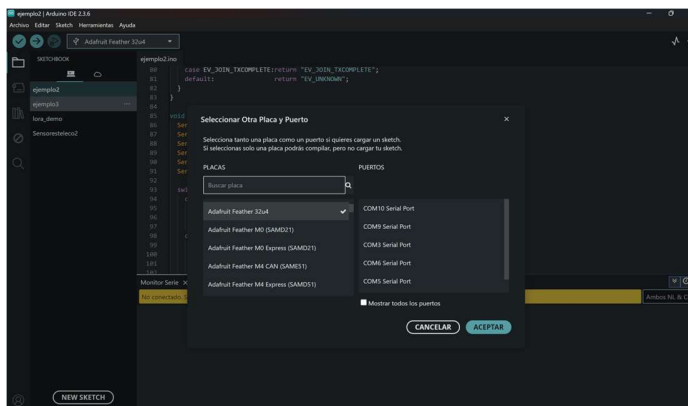


Figura 4: Selección de placa y puerto

4.2 Librerías requeridas

Instalar desde “Library Manager” (Herramientas → Administrar bibliotecas):

- MCCI LoRaWAN LMIC library (LMIC).

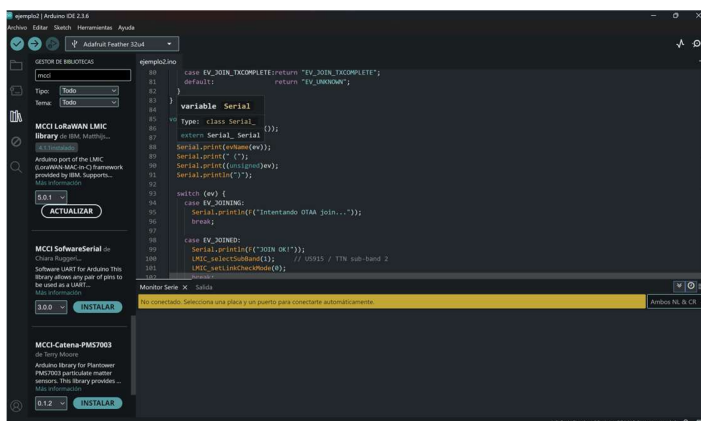


Figura 5: Instalación de librerías en Library Manager

5. Código del nodo LoRa (firmware)

Esta sección incluye el código principal del nodo. Ajusta DEVEUI/APPEUI/APPKEY según TTN.

```
#define CFG_us915 1
#define CFG_sx1276 1
#define LMIC_FAILURE_TO_SERIAL 0
#define LMIC_USE_INTERRUPTS 0

#include <lmic.h>
#include <hal/hal.h>
#include <SPI.h>
#include <DHT.h>

#define LDR_PIN A0
#define DHTPIN 2
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);
#define IR_PIN 3 // pin OUT del sensor IR

// APPEUI/DEVEUI LSB (invertidos), APPKEY normal
static const u1_t PROGMEM APPEUI[8] = { 0,0,0,0,0,0,0,0 };
static const u1_t PROGMEM DEVEUI[8] = { 0xBD,0x4D,0x07,0xD0,0x7E,0xD5,0xB3,0x70 };
static const u1_t PROGMEM APPKEY[16] = {
0x0D,0x70,0x5F,0x54,0xA6,0x4F,0xAA,0x4F,0xE6,0xDB,0x12,0xFC,0xF4,0xA6,0xD9,0x49 };

void os_getArtEui (u1_t* buf) { memcpy_P(buf, APPEUI, 8); }
void os_getDevEui (u1_t* buf) { memcpy_P(buf, DEVEUI, 8); }
void os_getDevKey (u1_t* buf) { memcpy_P(buf, APPKEY, 16); }

const lmic_pinmap lmic_pins = {
    .nss = 8,
    .rxtx = LMIC_UNUSED_PIN,
    .rst = 4,
    .dio = {7, 5, LMIC_UNUSED_PIN}
};

static osjob_t sendjob;
const unsigned TX_INTERVAL = 10;

void do_send(osjob_t*) {
    if (LMIC.opmode & OP_TXRXPEND) return;
```

```

int raw = analogRead(LDR_PIN);
uint8_t luz = (uint8_t)constrain(raw / 10, 0, 255);
uint8_t irState = digitalRead(IR_PIN); // 0 = detectado, 1 = libre

float h = dht.readHumidity();
float t = dht.readTemperature();

if (!(h == h) || !(t == t)) { h = 0; t = 0; } // NaN check sin isnan()

int16_t t100 = (int16_t)(t * 100.0f);
uint16_t h100 = (uint16_t)(h * 100.0f);

// Payload: [LDR][IR][T_hi][T_lo][H_hi][H_lo]
uint8_t payload[6];

payload[0] = luz;
payload[1] = irState;
payload[2] = (t100 >> 8) & 0xFF;
payload[3] = t100 & 0xFF;
payload[4] = (h100 >> 8) & 0xFF;
payload[5] = h100 & 0xFF;

LMIC_setTxData2(1, payload, sizeof(payload), 0);
}

void onEvent(ev_t ev) {
    if (ev == EV_JOINED) {
        LMIC_setLinkCheckMode(0);
        do_send(&sendjob);
    } else if (ev == EV_TXCOMPLETE) {
        os_setTimedCallback(&sendjob, os_getTime() + sec2osticks(TX_INTERVAL), do_send);
    }
}

void setup() {
    delay(3000);
    Serial.begin(9600);
    pinMode(IR_PIN, INPUT);

    dht.begin();
    os_init();
}

```

```

LMIC_reset();

LMIC_selectSubBand(1);      // sub-band 2
LMIC_setDrTxpow(DR_SF7, 14);
LMIC_setClockError(MAX_CLOCK_ERROR * 10 / 100);

LMIC_startJoining();
}

void loop() {
  os_runloop_once();
}

```

5.1 Estructura del payload

Se envían 5 bytes para todos los sensores.

- Byte 0: LDR (0–255)
- Byte 1: IR (0/1)
- Bytes 2–3: Temperatura (int16, °C×100)
- Bytes 4–5: Humedad (uint16, %×100)

Esquema del Payload LoRaWAN

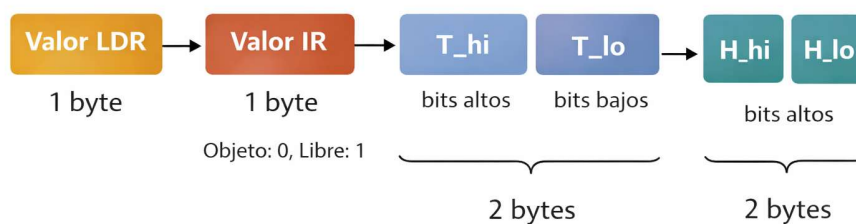


Figura 6: Esquema de payload.

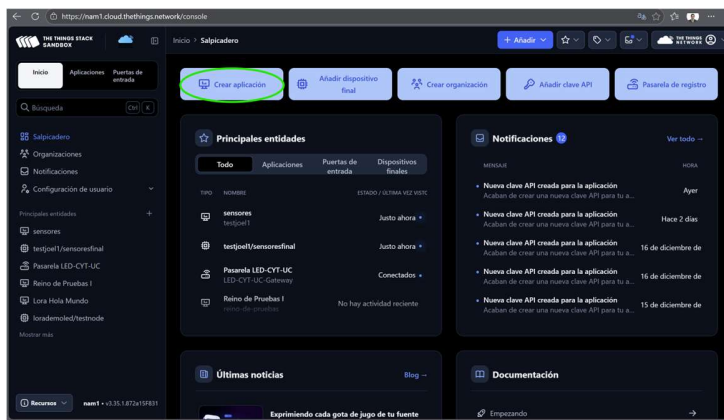
6. Configuración en The Things Network (TTN)

Esta sección documenta la creación de la aplicación, registro del dispositivo OTAA y configuración del decoder.

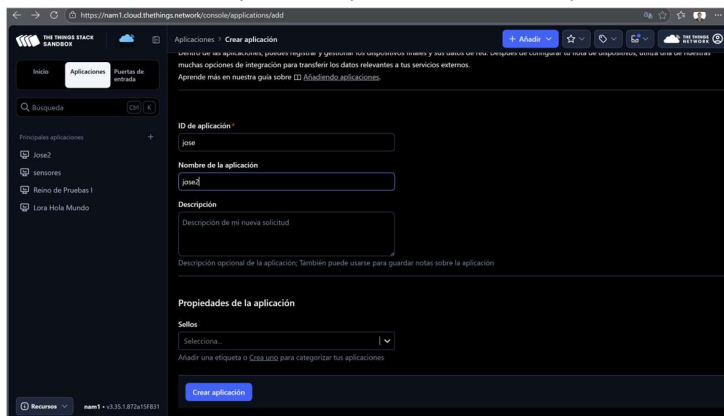
6.1 Crear aplicación

Accedemos a la pagina oficial de TTN <https://id.thethingsnetwork.org/> y nos logueamos con el usuario habilitado. Luego seguimos los pasos a continuación

1) Crear una aplicación.

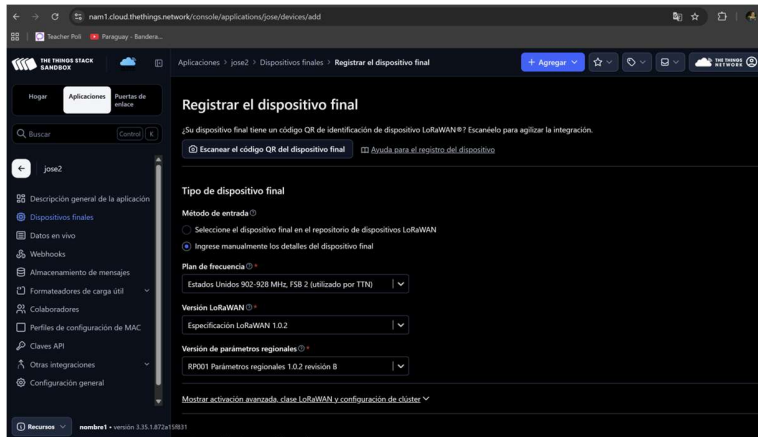


2) Insertar el id de aplicación y el nombre de la aplicación

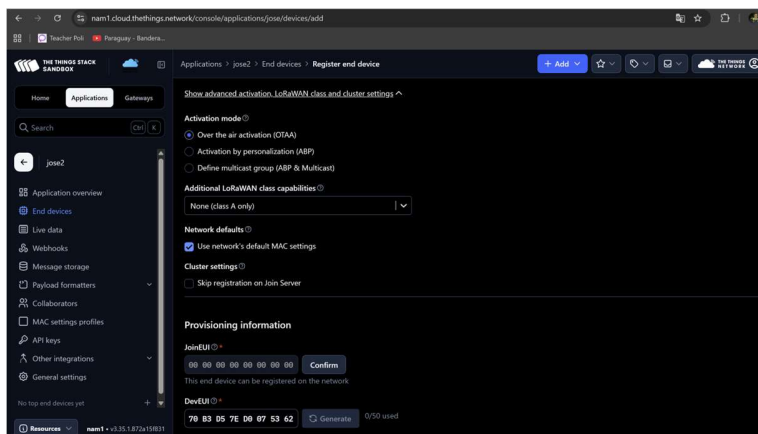


6.2 Registrar dispositivo (OTAA)

1) Agregar dispositivo final.



2) Activation mode: OTAA.



3) Copiar DevEUI / JoinEUI / AppKey.

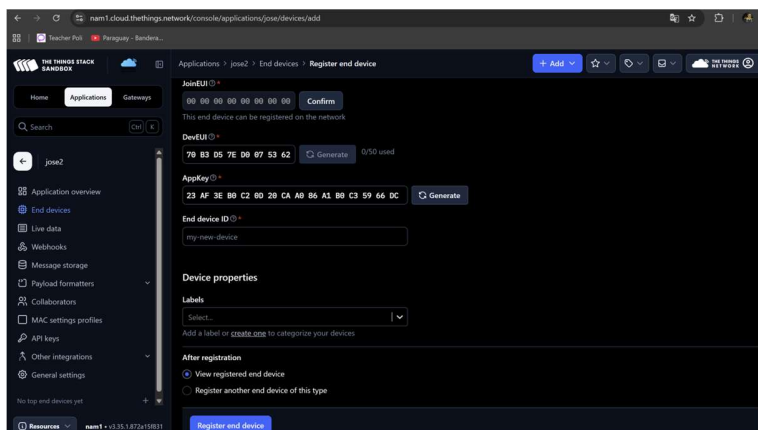


Figura 8: Registro del dispositivo.

6.3 Payload Formatter (Uplink Decoder)

Una vez registrado el dispositivo ingresamos a la pestaña PAYLOAD FORMATTERS y seleccionamos CUSTOM JAVASCRIPT FORMATTER, pegamos dentro el código para nuestro decoder:

```
function decodeUplink(input) {  
  return {  
    data: {  
      luz: input.bytes[0],  
      ir: input.bytes[1],  
      temp_c: ((input.bytes[2] << 8) | input.bytes[3]) / 100,  
      hum_rh: ((input.bytes[4] << 8) | input.bytes[5]) / 100  
    }  
  };  
}
```

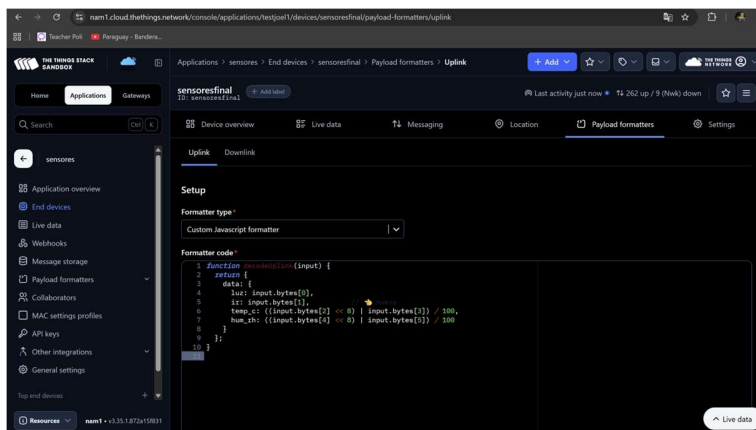
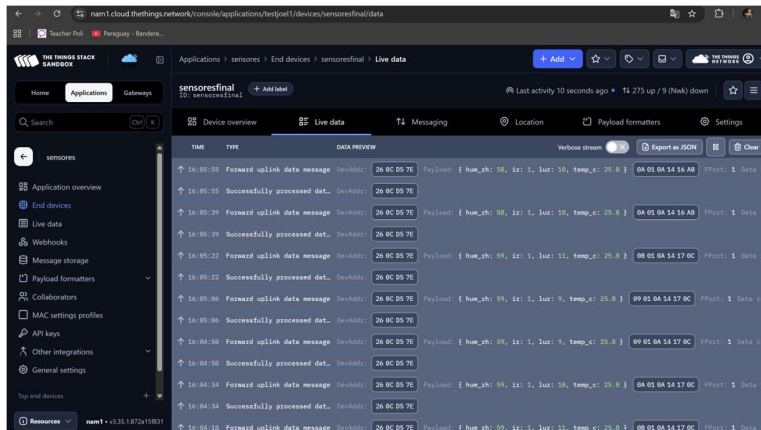


Figura 9: Configuración del payload formatter

6.4 Recepción de paquetes TTN

Una vez finalizado el paso anterior vamos a la pestaña de LIVE DATA y debemos visualizar los paquetes recibidos por el Gateway TTN como se muestra en la siguiente imagen:

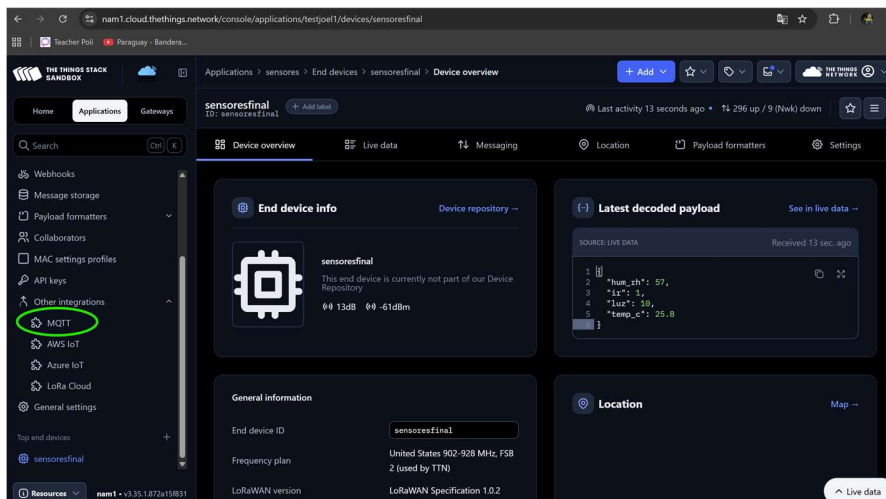


7. Integración MQTT

Utilizamos MQTT para consumir los uplinks desde TTN (broker TTN) y reenviarlos/consumirlos desde otros sistemas.

7.1 MQTT en TTN

En TTN Console → Integrations → MQTT: identificar Host, Puerto (TLS 8883), Username y API Key.



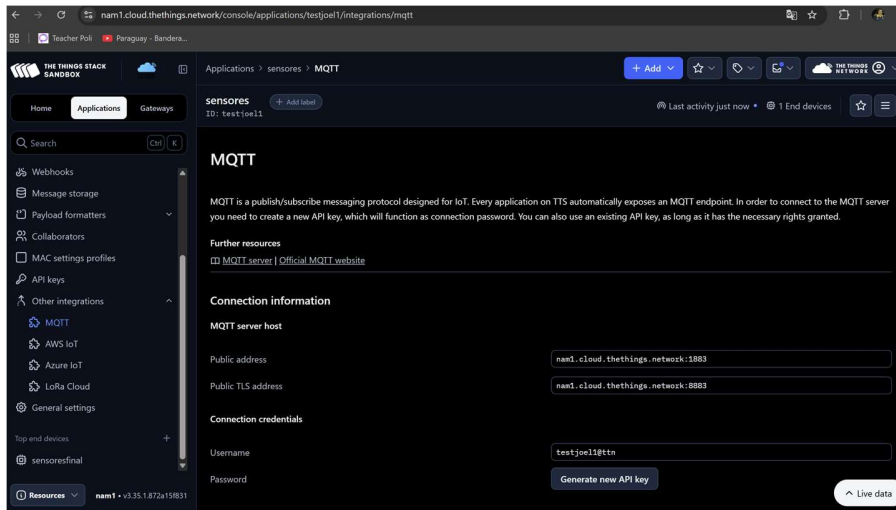


Figura 10: Pantalla de integración MQTT en TTN

Importante: darle clic a **generate new api key** para utilizar mas adelante.

7.2 Topics recomendados

Uplink (por dispositivo): `v3/<app>@<tenant>/devices/<device_id>/up`

Uplink (todos los dispositivos): `v3/<app>@<tenant>/devices/+up`

8. Instalación y configuración de Node-RED (Windows 11)

Esta sección describe la instalación de Node.js, Node-RED y el Dashboard.

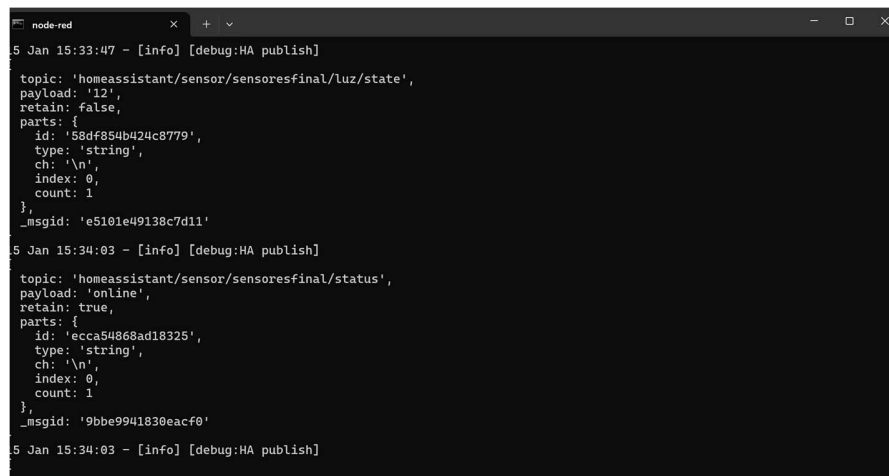
8.1 Instalar Node.js (LTS)

- 1) Descargar Node.js LTS.
- 2) Instalar con opciones por defecto.
- 3) Abrir el comando CMD y verificar con: `node -v` y `npm -v`.

8.2 Instalar Node-RED

Ejecutar en CMD: `npm install -g --unsafe-perm node-red`

Luego iniciar con: `node-red`

A screenshot of a terminal window titled 'node-red'. The terminal shows three MQTT messages being published. The first message has a topic 'homeassistant/sensor/sensoresfinal/luz/state', a payload of '12', and a retain flag of false. The second message has a topic 'homeassistant/sensor/sensoresfinal/status', a payload of 'online', and a retain flag of true. The third message is identical to the second. Each message is preceded by a timestamp and log level: '5 Jan 15:33:47 - [info] [debug:HA publish]', '5 Jan 15:34:03 - [info] [debug:HA publish]', and '5 Jan 15:34:03 - [info] [debug:HA publish]'. The messages are formatted as JSON objects with fields for topic, payload, retain, parts (id, type, ch, index, count), and _msgid.

```
node-red
5 Jan 15:33:47 - [info] [debug:HA publish]
topic: 'homeassistant/sensor/sensoresfinal/luz/state',
payload: '12',
retain: false,
parts: {
  id: '58df854b424c8779',
  type: 'string',
  ch: '\n',
  index: 0,
  count: 1
},
_msgid: 'e5101e49138c7d11'
5 Jan 15:34:03 - [info] [debug:HA publish]
topic: 'homeassistant/sensor/sensoresfinal/status',
payload: 'online',
retain: true,
parts: {
  id: 'ecca54868ad18325',
  type: 'string',
  ch: '\n',
  index: 0,
  count: 1
},
_msgid: '9bbe9941830eacf0'
5 Jan 15:34:03 - [info] [debug:HA publish]
```

Figura 12: Node-RED ejecutándose

8.3 Instalar paquetes en Node-RED

Ingresa en el navegador a node red con `http://localhost:1880/`, luego ingresa en la pestaña derecha superior al menú y clic en Manage palette.

Instalar desde Manage palette: `node-red-node-mqtt` y `node-red-dashboard`.

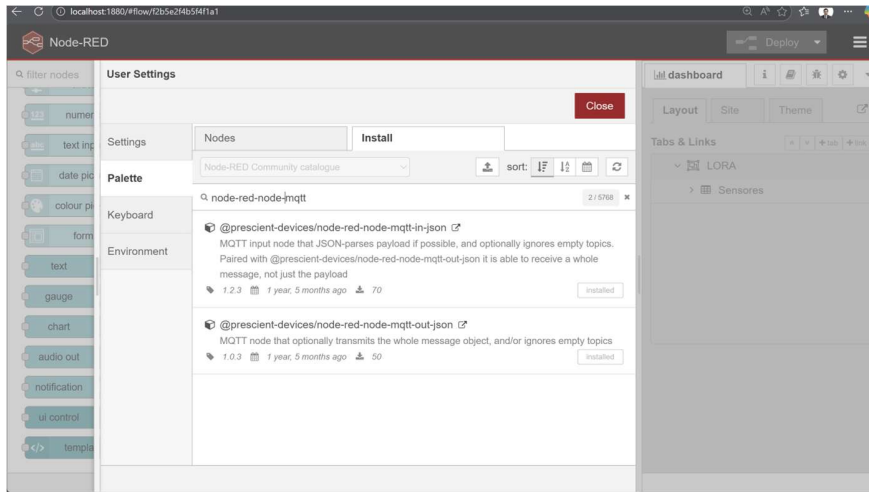


Figura 13: Instalación de paquetes.

9. Dashboard en Node-RED

Se crea un Tab “LORA” y un Group “Sensores”. Se agregan gauges para temperatura/humedad/luz y un indicador para IR, más un chart histórico.

Figura 14: Layout del dashboard.

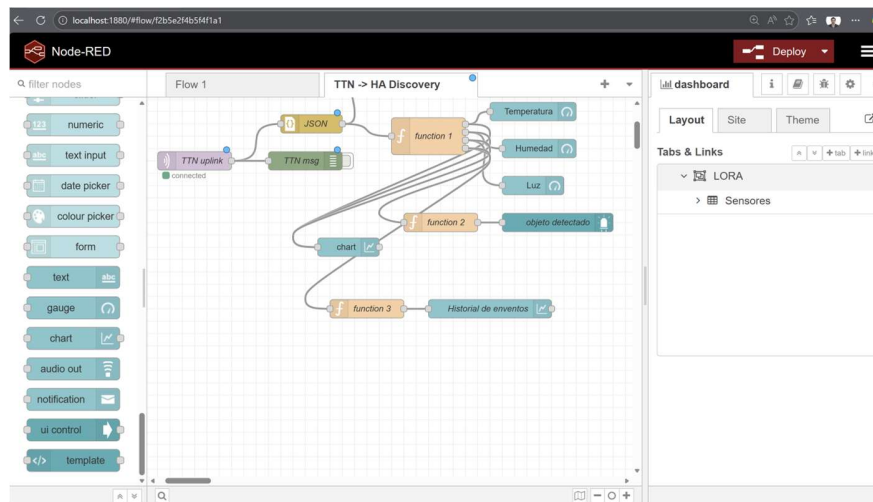


Figura 15: Dashboard final.

